Transferring Atomic Arbitrage Principles to Multi-chain Arbitrage Path Discovery and Execution

Roberto Brera Nick Felix Taejun Seo

May 10, 2025

Abstract

Price discovery in the Decentralized Finance (DeFi) ecosystem relies on effective arbitrage activity vis-a-vis an arbitrary number of assets (tokens) hosted across multiple chains. The proliferation of Layer-2 protocols (chains) have added much more complexity to arbitrage activity, often resulting in wide dissonance in the exchange rates of some esoteric token pairs hosted across multiple chains. This study makes two major contributions to the body of research in multi-chain arbitrage. First, we present a practical framework for implementing theoretical guidance on optimizing complex atomic arbitrage. Second, we make practical recommendations for extending the implementation framework to the multi-chain arbitrage context.

1 Introduction

Like in traditional finance, price discovery in decentralized finance relies on arbitrageurs effectively acting upon price discrepancies of an arbitrary number of asset pairs across multiple exchanges. Increasing trading volumes on decentralized exchanges (DEX) hosted on Layer 2 (L2) rollups have, in many cases, exacerbated price fragmentation and present new opportunities for cross-chain arbitrage between rollups or between rollups and a L1 centralized exchange (CEX).

Given the computational complexity of finding profitable arbitrage opportunities across a potential vast search space, previous works have emphasized careful algorithmic implementations. Wang et al. (2022) first introduced the concept of cyclic arbitrage opportunities in the blockchain, although their implementation only went so far as to traverse triangular cyclic arbitrage opportunities containing ETH. McLaughlin, Kruegel, and Vigna (2023) introduced more algorithmic sophistication by applying cycle detection algorithms to search for profitable cyclic arbitrage opportunities.

Subsequently, the Moore-Bellman-Ford (MBF) algorithm (and its variations) has emerged as the preferred algorithmic foundation for finding arbitrage opportunities. To our knowl-

edge, Zhou et al. (2021) was the first to apply the MBF combined algorithm to find arbitrage loops. However, subsequent work found that the MBF algorithm was significantly limited: it is able to recognize only a limited number of arbitrage opportunities, cannot specify the starting token of the detected arbitrage loops, and is unable to detect non-loop arbitrage paths between an arbitrage pair of tokens. In response, Zhang, Yan, et al. (2024) presents a modified methodology based on a modified MBF algorithm (MMBF) that uses a "line graph" with pairs of tokens as vertices in conjunction with the traditionally formulated graph consisting of tokens as its vertices. With some additional minor modifications, the methodology of Zhang, Yan, et al. (2024) is shown to be superior to prior formulations of the detection of MBF-based arbitrage opportunities.

However, we identify and resolve several important gaps in the literature. First, the aforementioned detection algorithms have been mostly presented from a purely theoretical standpoint, and previous works have provided little practical guidance on how to integrate them within a broader framework that accounts for the architecture-specific aspects of the blockchains we want to arbitrage. We address this gap by providing practical guidance on how to embed this promising theoretical foundation within a multithreaded system for detecting and executing atomic arbitrage trades in the single-chain setting.

In particular, we provide an end-to-end systems implementation that consists of four main modules: an online listener that maintains an updated blockchain state, an offline data processing unit that refines the state into an efficiently-processable graph, an offline solver that runs the main detection algorithms, and finally an online module for broadcasting the output transactions and handling the gas-bidding process. This modularization will provide a concrete roadmap for developing and iteratively improving an arbitrage-exploiting system.

Second, previous works' theoretical articulation and empirical demonstration of the aforementioned detection algorithms have been restricted to the single-chain environment. In response, we formulate a theoretical framework for applying the core principles found in single-chain algorithms to a multi-chain context. This will allow the system in the previous part to be extended for the more complex multi-chain arbitrage environment, potentially enabling the exploitation of more profitable opportunities.

2 Theoretical Background

In the following, we summarize the most salient findings and methodologies of previous work concerning the detection of arbitrage opportunities in the Web3/blockchain context. We highlight a theoretical framework for evaluating arbitrage strategies, summarize the major classical and frontier algorithms for detecting and evaluating opportunities, and illustrate a closed-form mathematical model for optimizing trade size for a given arbitrage opportunity.

2.1 Single-Chain Atomic Arbitrage in DeFi

Atomic arbitrage refers to the execution of a series of trades within a single blockchain transaction to exploit price discrepancies between decentralized exchanges (DEX). The typical sequence consists of the acquisition of capital, the execution of cyclical trades, and the repayment of loans. The atomicity of the single-transaction strategy enables arbitrageurs to leverage flash loans – uncollateralized loans that must be repaid within the same transaction. Assuming a well-designed execution code, all legs should settle atomically (or else revert entirely), thereby allowing the arbitrageur to capture the price discrepancy within that block, with the potential downside capped by the maximum expendable gas fees. Thus, these flash loans should virtually carry no default risk or exposure to interim price risk.

Prior research on single-chain DEX markets (e.g. Uniswap or Curve on Ethereum) shows that such atomic arbitrage opportunities arise when exchange rates across pools diverge and arbitrageurs quickly eliminate these inefficiencies (Öz et al. 2025). A classic example is a triangular swap: an arbitrageur borrows assets (via a flash loan), then trades through a cycle of pools (e.g., Token A $\rightarrow$ Token B $\rightarrow$ Token C $\rightarrow$ back to Token A) such that the starting asset quantity increases by the end of the cycle. These strategies are highly competitive; low barriers to entry (no capital requirement due to flash loans) lead to thin margins and intense bidding wars in transaction fee markets. This will later serve as the main motivation for extending these strategies to the less tractable multi-chain setting.

2.2 Graph-Based Cycle Detection and Bellman-Ford

To systematically discover single-chain arbitrage, researchers and practitioners model the DEX ecosystem as a directed weighted graph, where vertices represent tokens and directed edges represent swap trades in a specific pool. Specifically, we model the DEX ecosystem as a directed graph $G = (V, E)$ where:

- Each vertex $T_i \in V$ represents a token,
- Each directed edge $(i \rightarrow j) \in E$ encodes the net exchange rate:

$$R_{i\rightarrow j} \;=\; \frac{\text{units of } T_j \text{ received}}{\text{units of } T_i \text{ swapped}} \quad \text{(net of fees and slippage)},$$

  In order to be able to later apply the Bellman-Ford negative-cycle-detection algorithm on the graph, we transform to negative additive weights via the trick

$$w_{i\rightarrow j} \;=\; -\ln\big(R_{i\rightarrow j}\big)$$

Given the graph graph $G = (V, E)$ defined above, a cycle

$$C : T_{i_0} \rightarrow T_{i_1} \rightarrow \cdots \rightarrow T_{i_{k-1}} \rightarrow T_{i_k} = T_{i_0}$$

is profitable exactly when $\prod_{m=0}^{k-1} R_{i_m \to i_{m+1}} > 1$. Equivalently, one shows:

$$\prod_{m=0}^{k-1} R_{i_m \to i_{m+1}} > 1 \quad \Longleftrightarrow \quad \sum_{m=0}^{k-1} \ln\big(R_{i_m \to i_{m+1}}\big) > 0 \tag{1}$$

$$\Longleftrightarrow \quad \sum_{m=0}^{k-1} \big[-\ln\big(R_{i_m \to i_{m+1}}\big)\big] < 0 \tag{2}$$

$$\Longleftrightarrow \quad \sum_{m=0}^{k-1} w_{i_m \to i_{m+1}} < 0 \tag{3}$$

Thus, detecting any negative-weight cycle in G (with $w = -\ln R$) is exactly the same as finding a profitable cyclic arbitrage. The former is a well-known problem in graph theory and algorithms.

The classical solution is to run the Bellman–Ford shortest path algorithm (or a similar negative-cycle detection method) on this graph. By introducing a fictitious super-source vertex connected to all tokens with 0-weight edges, one can ensure any negative cycle is reachable from the source. After the usual $|V| - 1$ relaxation rounds, an additional relaxation that still yields improvement signals a reachable negative cycle (i.e., an arbitrage loop). In practice, Bellman–Ford can successfully identify at least one cycle where the product of exchange rates exceeds 1, corresponding to a profitable arbitrage opportunity. This approach (sometimes called the Bellman-Ford or BF arbitrage detector) became a standard starting point for on-chain arbitrage bots and academic studies of DEX pricing.

The below graph illustrates an example arbitrage path that traverses 3 of 4 tokens represented in the global state. Note that the arbitrage path will not necessarily traverse all 4 tokens shown if it is not profitable.

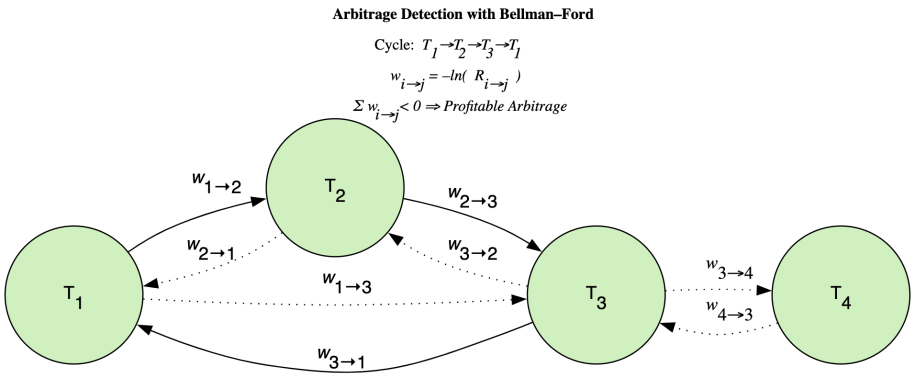

Figure 1: Generalized Bellman–Ford arbitrage detection. The solid arrows denote the main profitable cycle $T_1 \to T_2 \to T_3 \to T_1$, while the dotted arrows indicate other available (non-arb) swap directions and the additional token T_4 connected only by non-cycle edges. Each directed edge weight is defined by $w_{i \to j} = -\ln R_{i \to j}$.

However, a naïve out-of-the-box Bellman–Ford implementation has significant limitations in the fast-paced DeFi context. First, the standard algorithm terminates as soon as it finds any negative cycle, often returning an arbitrary cycle without regard to its expected profit. Specifically, the standard form of the algorithm has no built-in mechanism to maximize for the *magnitude* of the negative cycles it detects. In a large token graph, there may be dozens of concurrent arbitrage loops per block; a vanilla BF might stop at a trivial low-profit cycle while missing a far more lucrative opportunity.

Second, the input graph that Bellman-Ford receives, as constructed above, contains only partial costs, as it does not account for the gas fees incurred by the execution of longer cycles. This implies an edge-case cycle with a tiny percentage gain could be flagged even if, in absolute terms, it yields a negligible profit, or even a loss once gas is paid (McKinnon 2022). Thus, longer cycles will need to be penalized. Some candidate strategies include baking in the estimated gas fees directly in the weight metrics, bounding the maximum acceptable length for a cycle, or simply simulating execution of candidate cycles directly in the local EVM in order to discern the profitable ones from the non-profitable.

Third, out-of-the-box BF does not enumerate all cycles – it simply detects the existence of at least one arbitrage loop. One tangible, but potentially inefficient, solution is to recursively remove an edge representing the detected cycle and run the BF again, thereby enumerating all cycles one by one (McKinnon 2022; DeFigueiredo 2022).

Fourth, BF is inherently inefficient, as it runs $|V|-1$ consecutive rounds of relaxations. These operations are hard to parallelize, imposing a severe limitation on the size of the graphs that the algorithm would be able to process within the highly restricted block times (Layer 2 blocks can last for as little as two seconds).

In the following sections, we address these limitations by proposing tangible mitigation strategies.

2.3 Multi-Chain Arbitrage and Cross-Chain Strategies

As DeFi expanded beyond a single blockchain, arbitrage opportunities emerged across multiple chains (e.g., Ethereum, BSC, Polygon), where the same asset can trade at different prices on different networks. Cross-chain arbitrage exploits these inter-chain price discrepancies, but it introduces fundamental challenges absent in single-chain atomic scenarios.

2.3.1 Non-atomicity in multi-chain arbitrage

In particular, trades spanning distinct blockchains cannot be executed atomically within one transaction. Any cross-chain arbitrage must occur via separate transactions on each chain, often with a bridge transfer of assets in between – making it a non-atomic arbitrage susceptible to intermediate price movements and execution risk (Öz et al. 2025). This non-atomicity means that arbitrageurs face asynchronous execution: one leg of the arbitrage might be executed on one chain while the other leg is still pending (or the asset is in transit across a bridge). If prices shift unfavorably during that delay, the overall trade can fail or even incur losses.

Despite these risks, the fragmentation of liquidity across chains ensures that significant cross-chain price spreads do occur (Öz et al. 2025). Recent empirical studies have documented substantial cross-chain arbitrage activity. For example, Öz et al. (2025) identify over 260,000 cross-chain arbitrage transactions in a year across nine major blockchains. These arbitrages generated at least $9.5 million in total profit and often involved trading popular assets like WETH and USDC on Layer-2 networks vs. Ethereum mainnet. The observed cross-chain arbitrages can be categorized into two strategic paradigms: Sequence-Independent Arbitrage (SIA) and Sequence-Dependent Arbitrage (SDA) (Öz et al. 2025).

2.3.2 Sequence-Independent Cross-Chain Arbitrage (SIA)

In Sequence-Independent cross-chain arbitrage (SIA), an arbitrageur executes independent, opposite-direction trades on different chains without relying on any cross-chain transfer during the arbitrage (Öz et al. 2025). Essentially, the arbitrageur maintains float (inventory) of the relevant assets on each chain beforehand. When a price discrepancy arises (e.g., an asset is cheaper on Chain A than on Chain B), they buy on the underpriced chain and simultaneously sell on the overpriced chain, using their pre-positioned liquidity on both sides

(Öz et al. 2025). Because each trade is confined to its own chain's block, no inter-chain messaging or waiting is required at execution time. Hence, the crucial advantage of this setup is that the transactions can occur in synchrony – up to the block-time discrepancies of the blockchains – and thus more closely emulate the atomic case.

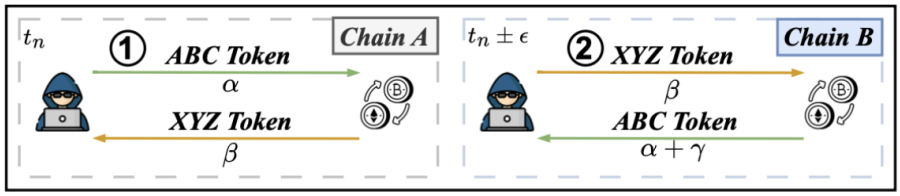

Figure 2: Sequence-Independent Cross-Chain Arbitrage (SIA). In a low-liquidity L2 pool (Chain A), an arbitrageur trades 1 ABC $\rightarrow$ 1 XYZ at price α then on Chain B immediately swaps 1 XYZ $\rightarrow$ $(\alpha + \gamma)$ ABC. Independently executed, this yields net profit γ. Figure by Öz et al. (2025)

.

This independence avoids bridge delays entirely since there is no need for bridging at all, greatly reducing exposure to price slippage due to timing. For instance, if 1 WETH is trading for 1000 USDC on a Layer-2 exchange while on Ethereum mainnet it trades for 1100 USDC, a SIA strategy might use 1000 USDC on the L2 to buy 1 WETH and simultaneously use 1 WETH on mainnet to sell for 1100 USDC. The net profit (100 USDC) is realized once the arbitrageur eventually rebalances their inventories (e.g., periodically bridging some profit to restore balances). The key advantage is thus the removal of bridge risk during the arbitrage, which otherwise could "defeat the purpose of arbitrage" by turning it into a speculative bet on future prices.

However, the trade-off is that SIA requires significant capital to be fragmented across all chains and assets involved. In practice, maintaining liquidity in every token on every chain is infeasible beyond a small subset; for instance, an arbitrage spanning 9 chains and 10 tokens would demand maintaining up to 90 separate liquidity positions. Thus, as a more practical compromise, arbitrageurs can employ collateralized cross-chain loans: rather than pre-funding every token, they hold a large stash of collateral (e.g. a stablecoin or ETH) on each chain and quickly borrow the specific asset needed for a given trade from a local money market. Thus, each chain requires just one pool of high-value collateral to tap into, and does not need to maintain a full inventory of all tradable assets. However, the arrangement still incurs overhead and complexity in loan management.

In the upcoming sections, we will propose a theoretical framework for handling this complex borrowing scheme and bridging collateral pools across chains. The goal will be to emulate the atomic case, modeling the exchange-rates offered across multiple chains in a

single general graph, to then delegate cross-chain liquidity provision and loan management to a specialized subroutine.

2.3.3 Sequence-Dependent Cross-Chain Arbitrage (SDA)

By contrast, Sequence-Dependent cross-chain arbitrage (SDA) uses bridge contracts or cross-chain swaps as part of the transaction sequence (Öz et al. 2025). In SDA, the legs of the arbitrage are executed in a dependent order: the output (profit) from the first trade (on Chain A) is transferred to Chain B via a bridge to fund the second trade (Öz et al. 2025). This strategy minimizes initial capital needs, because the trader can start with funds on only one chain and move value to the other as needed, rather than holding assets on both. [1]

The drawback is the latency and risk introduced by the bridge step: between the L2 purchase and the mainnet sale, the market could move or another arbitrageur could correct the price discrepancy. Empirically, SDA trades are indeed riskier and sometimes fail (e.g., if the price gap closes while funds are en route). Öz et al. (2025) observe that only about one-third of the cross-chain arbitrages detected in their data set involved bridging, indicating that many arbitrageurs prefer the safer SIA approach despite its higher capital requirement.

In practice, the SIA strategy is significantly more prevalent, with much of 67.63% of cross-chain arbitrage not using a bridge . The prevalence of SIA over SDA can be attributed to the time-sensitive nature of arbitrage strategies — waiting for funds to be transferred over bridges could render arbitrage opportunities unprofitable. Therefore, in our work we choose to build on the SIA approach as we believe that liquidity provision is a more correctable pitfall than potentially unpredictable price slippages.

2.4 CFMM Closed-Form Solution

In our graph-modeling approach, we compute the exchange rates between some arbitrary tokens X, Y by simply taking the relative ratio $\frac{y}{x}$ of their respective reserves (x, y). This is known as the *mid-price* and is a perfectly reasonable assumption to make in the context of detecting potentially profitable arbitrage cycles. However, when we would then actually trade the tokens, the effective price we get is crucially dependent on trade size, especially when the trade size is large compared to the reserves.

Therefore, in this section we show the derivation for a convenient closed-form formula to optimize the trade size of an arbitrary cycle.

[1] Using the earlier example, an SDA execution might begin by buying 1 WETH on the cheaper L2 (spending 1000 USDC), then bridging that WETH to Ethereum mainnet, and finally selling it for 1100 USDC on mainnet. The outcome (1100 USDC on mainnet, 1000 USDC spent on L2) is the same net profit of 100 USDC, but it is achieved with capital initially only on the L2 side.

In a constant-product CFMM (e.g., Uniswap-style pool), ignoring fees for simplicity, the output Δy from input Δx follows a concave curve

$$\Delta y \;=\; y \;-\; \frac{x\,y}{x+\Delta x}\,,$$

so the *effective price* $\Delta y/\Delta x$ depends on Δx itself, especially when Δx is large relative to the reserves (x, y). Thus, once we identify a cycle

$$T_0 \;\to\; T_1 \;\to\; \cdots \;\to\; T_0,$$

we must also solve a concave optimization problem—maximizing the composite rational function given by chaining each pool's $\Delta x \mapsto \Delta y$ mapping and subtracting gas costs—to find the true profit-maximizing order size.

A straightforward practical approach is to run a *binary search* on Δx for each candidate cycle: for each trial size, simulate the full loop, subtract estimated gas, and converge in ~ 10 iterations to within a tight tolerance. In production this meant performing ≈ 100 such CFMM evaluations per block (hundreds of milliseconds even on parallel hardware). Ultimately, however, deriving and implementing the *closed-form solution* for the optimal Δx in a concatenated CFMM chain proved far more efficient, eliminating the per-block binary-search overhead. Consider the following derivation of the closed-form solution by Lipton and López de Prado (2020).

We consider a constant-product automated market maker (CFMM) with reserves (R_1, R_2) for token pair (t_1, t_2) and trading fee parameter $0 < \gamma \leq 1$ (so the trader pays an effective fee fraction $1 - \gamma$). We derive the output function, marginal price, and the optimal arbitrage order size for general cyclic paths.

2.4.1 Single Pair Trade

First, consider a single-pair trade. Inputting a quantity $\Delta_1 > 0$ of token t_1 to the CFMM yields:

$$\Delta_2(\Delta_1; R_1, R_2) \;=\; \frac{\gamma\, R_2\, \Delta_1}{R_1 + \gamma\, \Delta_1}\,. \tag{4}$$

The corresponding *marginal price* of t_2 in units of t_1 after supplying Δ_1 is

$$\frac{\mathrm{d}\Delta_2}{\mathrm{d}\Delta_1} = \frac{\gamma\, R_1\, R_2}{\big(R_1 + \gamma\, \Delta_1\big)^2}\,. \tag{5}$$

2.4.2 Extension to Arbitrary Multi-Hop Paths

Next, consider an arbitrary multi-hop path consisting of n successive CFMMs trading $t_1 \to t_2 \to \cdots \to t_n$ with reserves $R_{i,i+1}$ on each hop (t_i, t_{i+1}). Let Δ_1 be the input on the

first market, and define Δ_k to be the output of the k-th hop. One shows by induction that

$$\Delta_n(\Delta_1; R) = \frac{\Delta_1 \, \gamma^n \, \prod_{i=1}^{n-1} R_{i,i+1}}{\prod_{i=1}^{n-1} R_{i+1,i} \; + \; \Delta_1 \, \sum_{i=1}^{n} \gamma^i \Big(\sum_{j=1}^{i-1} R_{j,j+1} \Big) \Big(\sum_{j=i+1}^{n} R_{j,j-1} \Big)} \,, \tag{6}$$

and its marginal price is

$$\frac{\mathrm{d}\Delta_n}{\mathrm{d}\Delta_1} = \frac{\gamma^n \, \prod_{i=1}^{n-1} R_{i,i+1} R_{i+1,i}}{\Big(\prod_{i=1}^{n-1} R_{i+1,i} \; + \Delta_1 \sum_{i=1}^{n} \gamma^i (\sum_{j=1}^{i-1} R_{j,j+1})(\sum_{j=i+1}^{n} R_{j,j-1}) \Big)^2} \,. \tag{7}$$

2.4.3 Optimal Arbitrage Quantity

For a cyclic arbitrage path $t_1 \to t_2 \to \cdots \to t_n \to t_1$, the net profit when inputting Δ_1 of t_1 is

$$\Pi(\Delta_1) = \Delta_{n+1}(\Delta_1; R) - \Delta_1 \,, \tag{8}$$

where Δ_{n+1} denotes the final output back in token t_1. Maximizing $\Pi(\Delta_1)$ yields the closed-form optimal input:

$$\Delta_1^* = \frac{\sqrt{\gamma^n \, \prod_{i=1}^{n} R_{i,i+1} \, R_{i+1,i}} - \prod_{i=1}^{n} R_{i,i+1}}{\sum_{i=1}^{n} \gamma^i \Big(\prod_{j=1}^{i-1} R_{j,j+1} \Big) \Big(\prod_{j=i+1}^{n} R_{j+1,j} \Big)} \,, \tag{9}$$

which simplifies the brute-force (binary–search) approach and runs in closed form.

3 Methodology

As previously discussed in section 2, while the Bellman–Ford approach forms the fundamental building block for arbitrage detection, it must be adapted to DeFi's real-time requirements. Specifically, we seek to design an algorithm that: (1) in general, balances a) thoroughness (finding as many profitable cycles as possible) with b) speed and practicality in a real-time trading environment; and (2) is extendable to a multi-chain environment.

The literature suggests a spectrum of optimizations for pure BF-based negative cycle search, such as improved multi-cycle enumeration and ranking methods (Zhang, Li, et al. 2024), to algorithmic innovations that broaden the search space while managing complexity (pruning by liquidity/length, specialized graph transformations, and profit-driven objective functions). Here, we provide a framework for designing and evaluating potential implementation choices for multi-chain arbitrage.

3.1 Design Requirements

We formulate the following principles in considering our implementation, organized by descending level of importance. Moreover, we explain how we have addressed each one of thee requirements in our implementation, and how previous work has dealt with them.

Requirement 1 (Time Complexity). The algorithm must find profitable arbitrage paths, if any, as quickly as possible within a strict upper bound of the block time of the blockchain.

The algorithm runtime is subject to a strict upper bound equal to the block time of the native blockchain (e.g., 12 seconds in Ethereum). Should it fail to produce a profitable arbitrage path within this bound, and the blockchain state is updated to reflect the next block, the strategy practically becomes obsolete as exchange rates would have moved significantly. In practice, satisfying this requirement would necessitate limiting the number of edges that the algorithm would have to traverse in the graph, as the algorithm primarily executes edgewise iterations. We suggest two realistic strategies for achieving a practical runtime.

First, it is crucial to minimize the data load that has to be processed by the cycle-detection algorithm at the source, by enforcing strict filters on the liquidity pools and DEXes that are gathered during the first screening phase. In our implementation, we have set minimum limits on the reserve size in ETH of each liquidity pool, since too-small liquidity pools are practically impossible to arbitrage at sufficiently large trade sizes as a consequence of the CFMM explained previously. Additionally, we could iteratively update a blacklist of "dead tokens" that are not arbitrageable due to their contracts having become inactive.

Second, we can prune the graph. For example, since the logical place to start the experiment is Uniswap V2 – possibly the most canonical DEX architecture – and most crucially this architecture enforces unique liquidity pools between any token pair, we already know upfront that only one single exchange rate will be offered for each edge. Because we always trade at a fee, an immediate consequence of this is the fact that 2-token cycles will never be profitable, which in turn allows us to prune from the graph any node whose outdegree is strictly less than 2 without impacting the overall arbitrage opportunities.

It is worth emphasizing that these data preprocessing procedures alone are largely responsible for increasing the efficiency of the overall system without even touching the implementation of Bellman-Ford. In our case, for example, they have brought down the value of the different tokens/nodes n from roughly 400,000 to under 5,000, with substantial implications for performance.

As a consequence of the nature of the problem, and our strict data screening process,

the graph will be sparse, thus presenting a low ratio of number of edges to the number of vertices, or $ev\ ratio = |E| : |V|$, which will help the efficiency of the cycle-detection algorithm.

In practice, we expect there to be a set of "critical vertices", denoted M where $M \subset V$, that represent the highly liquid tokens that form of the bedrock of the DeFi ecosystem. These vertices should have a very high in/out degree of approximately $|V|$ each, as each of these foundational tokens should be convertible to almost any other token. We anticipate that these critical vertices should represent assets such as BTC, ETH, SOL, USDC, and a few others. On the other hand, the remaining $O = V - M$, where $O \subset V$ vertices would represent "obscure" tokens, characterized by limited exchange liquidity vis-a-vis other tokens represented in O. Thus, these tokens would mostly only be exchangeable for tokens represented in M. Consequently, we should expect verticies in O to have in/out degrees of approximately $|M|$.

Third, we enforce a hard cap on cycle length to deter overly long arbitrage paths that would be both computationally expensive to execute (thereby incurring more gas fees) and would be unlikely to be found in a reasonable amount of time (because of the combinatorial explosions in the search space). To enforce this cap, we simply limit the number of Bellman–Ford relaxations to some fixed integer $k < |V|$. After the k-th relaxation, no further distance updates are considered, ensuring that the algorithm only detects only cycles of at most k edges. This is a significant improvement to Bellman-Ford as it limits the total number of relaxations from $|V| - 1$ to a mere fraction of that amount.

In fact, empirical studies of DEX arbitrage find that most profitable opportunities involve only a few hops (2–4 trades) due to gas costs and slippage increasing with longer paths. Thus, many arbitrage bots restrict attention to short cycles (triangular or quadrilateral loops at most) and ignore longer paths beyond a certain length bound. This dramatically limits the number of cycles to consider. For example, McKinnon (2022) reports pruning his Avalanche C-Chain arbitrage graph by filtering out low-liquidity or inactive pools, and then only searching cycles of length 4 or shorter, which were sorted by historical profit potential. By focusing on a small subset of likely profitable cycles, their system reduced detection time from hundreds of milliseconds to tens of milliseconds per block.

Another line of work separates the problem into two phases: enumeration of candidate cycles followed by runtime evaluation. By using Johnson's cycle-finding algorithm (Johnson 1975) to enumerate all simple cycles in the trading graph, one can pre-compute the cycle topology off-chain (assuming a relatively static pool graph). At runtime, the weight of each cycle (product of current exchange rates) is then calculated to check for negativity, and cycles can be ranked according to estimated profit. This cycle-enumeration approach, while potentially expensive, becomes practical if the graph structure is relatively static (tokens/pools don't change often). It enables exhaustive discovery of arbitrage loops and identification of the top-profit opportunity at the cost of heavy pre-processing.

While we did not implement this approach in the specifics, we have inherited from it the spirit of runtime evaluation. After computing any candidate cycle, we leveraged the local EVM execution of the Uniswap V2 router contract to run locally the exact same code that would run on the EVM, and thus to verify the profitablity of each single candidate cycle (excluding the trading fees).

Finally, it is crucial to emphasize in this closing part that the screening and graph construction phase cannot and should not be repeated at every new block. Successful implementations monitor only a subgraph of pools that have changed in the latest block, re-running the cycle detection on that subgraph or updating distances incrementally, rather than recomputing on the entire graph each time. In our case, it took roughly 20 minutes to construct the whole up-to-date graph of the Uniswap V2 DEX, and hence this would be impractical to re-execute at every single new block.

Requirement 2 (Detection of an arbitrary number arbitrage opportunities). The algorithm must be able to find an *arbitrary* number of negative cycles dynamically.

Second, the algorithm must be able to find an *arbitrary* number of negative cycles dynamically. Unlike the "lazy" version of the algorithm that terminates after identifying *any* negative cycle, we seek to design an algorithm that continues to search for more optimal negative cycles so long as the block timestep remains the same. Ideally, the algorithm would return all possible fee-aware candidates for arbitrage paths, which can then be processed by a separate gas-bidding algorithm, which we define below.

There are two pragmatic approaches to returning more than one negative-cycle arbitrage in a token-swap graph. The first is a delete-and-rerun strategy, as suggested by McKinnon (2022): after Bellman–Ford returns a cycle C, remove one arbitrarily chosen edge from C and rerun the algorithm on the reduced graph. Repeating this N times yields N cycles that must each differ by at least one edge. This runs in $O\big(N \cdot |V| \cdot |E|\big)$, is relatively trivial to implement, and guarantees diversity. This heuristic does not guarantee finding the most profitable cycles as it risks missing some more profitable loops, but in practice it has shown to be able to enumerate more profitable candidates than what might be expected from a 'lazy' BF run.

The second is an enumerate-and-filter approach, which incorporates cycle-ranking heuristics into the search. For instance, rather than picking a random edge to remove in each iteration, one might preferentially remove the least profitable edge or otherwise bias the search so that larger negative-weight (higher profit) cycles surface early. Still, as the number of tokens and pools grows, the potential cycles grow combinatorially, making exhaustive search infeasible.

A more generic approach may be to recast the search as an optimization problem: for example, Angeris et al. (2021) show that finding an arbitrage in a network of CFMM

(AMM) pools can be cast as a convex feasibility problem (or a mixed-integer convex program when including fixed gas costs), essentially checking if a profitable routing exists (Angeris et al. 2021). Their formulation includes arbitrage detection as a special case of an optimal trade routing problem, indicating an arbitrage loop is present if and only if the optimal solution yields more value than one's starting assets (Angeris et al. 2021).

Given that in our case we were operating on the simpler Uniswap V2, and had done extensive data preprocessing, we could afford to go by the first simpler approach proposed by McKinnon which still ran in reasonable time.

Requirement 3 (Maximization of the negative sum of the arbitrage cycle). We seek to maximize the negative sum of the arbitrage cycle. Our cycle-detection algorithm should seek to surface as consistently as possible the highest-margin arbitrage opportunities after accounting for gas costs.

Finding the single most profitable cycle corresponds to solving the longest simple negative cycle problem, which is NP-hard in general (Angeris et al. 2021). DeFigueiredo (2022) highlights that standard shortest-path algorithms will detect a negative cycle but not necessarily the best one, and notes that identifying the globally optimal arbitrage loop would require solving a difficult combinatorial problem.

Thus, for the purpose of this paper, we seek to fulfill this requirement by ensuring that the graph remains always small enough to allow for the efficient enumeration of all negative cycles.

Requirement 4 (Execution for Profit-Maximization). The algorithm must solve the concave optimization problem with minimal computational overhead.

The requirements listed above will enable the algorithm to reliably return a near-optimal negative-weight cycle / arbitrage path in the token-swap graph. However, the algorithm must also be able to determine how much of each token to trade to achieve near-optimal profits. The CFMM closed form solution from section 2.4 can be leveraged to fulfill this requirement. However, for our trades we anticipate the liquidity pool reserves to be large enough compared to our trade size in most of the cases, implying that we will not need to worry about trade size.

3.2 Algorithm Design

Given the above requirements, we design an end-to-end algorithm that consists of four main modules:

1. An online listener that maintains an updated state for the liquidity pool pairs for the DEX(es) being monitored.
2. An offline data processing unit that filters the liquidity pool pairs, and maintains an up-to-date pruned graph.
3. An offline solver that takes the pruned graph as input and runs the Bellman-Ford algorithm with the modifications described above to detect candidate arbitrage loops; A liquidity provision module that wraps cycles into transactions handling the loan procedure
4. An online module for broadcasting the transactions (possibly to the Flashbots system) and eventually handling the gas-bidding process.

We present the main pseudocode below:

Algorithm 1 End-to-End Single-Chain Arbitrage Framework, *Optimized*

Require: Factory contracts for all target DEXes, blacklists, thresholds

// Initialization
Initialize empty pool index $\mathcal{P}$
Subscribe to `PairCreated`, `Sync`, `Swap` events
while true **do**
    // 1. Online State Listener
    Wait for next on-chain event e or new block
    if e is PairCreated or Sync or Swap **then**
        Update or insert $\mathcal{P}[e.\texttt{pairAddress}]$
    end if
    // 2. Offline Data Processor (triggered per block)
    FilteredPools $\leftarrow \emptyset$
    for all pair p in $\mathcal{P}$ **do**
        if p passes liquidity, blacklist, and reserve filters **then**
            Add p to FilteredPools
        end if
    end for
    Build directed graph G from FilteredPools
    Prune G (min-degree, isolate removal, peg-collapses)
    Adjust edge rates by gas/slippage factor $\gamma \leq 1$
    Transform to weights $w(u \to v) \leftarrow -\ln(\gamma\, R_{u \to v})$
    Let $G' \leftarrow G$ with these weights
    // 3. Graph-Searcher & Transaction Builder
    Enqueued $\leftarrow \emptyset$
    for all negative cycle c detected in G' (Bellman–Ford) **do**
        Compute optimal trade size Δx^* and net profit
        if net profit > 0 **then**
            if c lies on single chain **then**
                Plan flash-loan + swap transaction
            else
                Partition c into chain-local segments
                for all segment s **do**
                    Prepare borrow–swap–bridge & debt-settle txns
                end for
            end if
            Simulate all planned transactions on a local EVM fork
            if simulation succeeds **then**
                Sign and add to Enqueued
            end if
        end if
    end for
    // 4. Submission
    for all tx in Enqueued **do**
        Submit tx on its target chain
    end for
end while

3.2.1 Gas Bidding Module

In the above implementation, we abstract away the optimal gas-bidding process. We present the randomized PGA gas-bidding strategy as a possible implemenation:

Algorithm 2 Randomized PGA Gas-Bidding Strategy

Require: Integer bounds $n_{\min}, n_{\max}$, fee bounds b, B, time window $[t_0, t_n]$, noise variance σ^2

Draw $n \sim \mathcal{U}[n_{\min}, n_{\max}]$
for $i = 0$ to n **do**
    $t_i \leftarrow t_0 + i \cdot \frac{t_n - t_0}{n}$
    $b_i \leftarrow b + i \cdot \frac{B - b}{n}$
    repeat
        $\tilde{t}_i \sim \mathcal{N}(t_i, \sigma^2)$
    until $\tilde{t}_i > \tilde{t}_{i-1}$ ▷ enforce monotonic times
    $t_i \leftarrow \tilde{t}_i$
    repeat
        $\tilde{b}_i \sim \mathcal{N}(b_i, \sigma^2)$
    until $\tilde{b}_i > \tilde{b}_{i-1}$ ▷ enforce monotonic bids
    $b_i \leftarrow \tilde{b}_i$
end for
for $i = 0$ to n **do**
    Wait until wall-clock time t_i
    Re-submit transaction with gas fee b_i
end for

Note also that we would also ideally continuously run a gas-bidding thread to make bids as paths are found. The gas-bidding thread will run for the duration of one block, and be re-initialized and run again from scratch upon the arrival of each new block:

Algorithm 3 Gas-Bidding Thread

```
Initialize an empty, time-sorted list of bid events r
while termination signal not received do
    if a new transaction tx is added to q then
        Compute its sequence of bid times {t_i} and bid values {b_i}
        Insert each event (t_i, (tx, b_i)) into the sorted list r
    end if
    if current time > r[0].t then
        Re-bid transaction r[0].tx with fee r[0].b
        Remove the head event r[0] from r
    end if
end while
```

The implementations above will serve as the jumping off point for our discussion of how a derivative of this algorithm can be implemented for multi-chain arbitrage (Section 3.3).

3.3 Non-atomic arbitrage algorithm for multi-chain opportunities

In this final subsection, we suggest a realistic high-level plan for amending the single-chain and atomic, cycle-detection and execution framework such that it can be extended to a sequence-independent, cross-chain setting. We present this amendment in three stages, each relaxing one of our earlier assumptions.

1. Building a Global Best-Rate Graph
We start by arguing that from a purely modeling and detection perspective the cross-chain scenario can be encapsulated in a single global best-rate graph. This will then enable us to port the previously developed offline graph searcher unit to this multi-chain context.

More concretely, in this setting we let the nodes still refer to the single tokens exactly as in the single-chain case. However, the edges now need not only encode the relative exchange rates, but it must also encode the particular chain on which any given exchange rate is offered. In a full augmented graph version, each swap pair - represented as an edge $e(X, Y)$ - would be mapped to a dictionary, which maps each blockchain to the best available exchange rate for (X, Y). (It is trivially the case that a blockchain where a swap between the tokens isn't feasible would not be included in the dictionary.) Next, for the sake of space complexity, we can keep just the best exchange found in any blockchain for the token pair. Equivalently, for the purpose of arbitrage detection, we can condense the edge weights between any two particular tokens to the optimal exchange rates offered across all the available chains.

More formally:

Let there be b blockchains $\{B_1, \ldots, B_b\}$ and a total of n distinct tracked tokens. For each token pair (X, Y), query on each chain B_i the m_i leading DEXes $\{D_{i,1}, \ldots, D_{i,m_i}\}$ for the best available swap rate between (X, Y):

$$R_{X\to Y}^{(B_i)} \;=\; \max_{D\in\{D_{i,1},\ldots,D_{i,m_i}\}} R_{X\to Y}^{(D)}$$

Next, for each token pair (X, Y) place in the global graph G a directed edge $X \to Y$ with scalar weight $R_{X\to Y}$ represented by the best exchange rate offered across all of the blockchains:

$$R_{X\to Y} \;=\; \max_{i\in\{1,\ldots,b\}} R_{X\to Y}^{(B_i)}$$

Additionally, record the originating chain B_j as additional edge data where

$$j \;=\; \arg\max_{i\in\{1,\ldots,b\}} R_{X\to Y}^{(B_i)}$$

This will be crucial in order to later trace back the correct blockchain on which to perform the swap between (X, Y).

Having now obtained a graph G of the same form as in the single-chain case, we can apply the same logic.

In short, we transform the edges of G via $w_{X\to Y} = -\ln R_{X\to Y}$ and then run our version of Bellman–Ford yielding some candidate profit cycle of the form

$$c = \big(T_0 \to T_1 \to \cdots \to T_k = T_0\big).$$

2. Partitioning into Chain-Local Swap Sequences

At this point, we have managed to obtain a candidate cycle as in the single-chain case. However, we cannot simply package it into a single atomic transaction since the edges may span across multiple different blockchains.

In order to break down the cycle into smaller segments such that each is contained within a single blockchain, we label each edge of c by its chain, and then split c into r contiguous, chain-homogeneous segments that we denote as

$$c = c^{(1)} \,\|\, c^{(2)} \,\|\, \cdots \,\|\, c^{(r)},$$

where each $c^{(j)}$ lies entirely on a single blockchain B_{i_j}. Assuming that the required liquidity on each segment is available, these r segments can be executed independently (and in synchrony up to the inherent block-time discrepancies between the blockchains), using the same spirit that we have seen in SIA. The next section will take care of the premise, proposing a scheme to provide the liquidity necessary for the trades.

3. Liquidity Provision via Collateralized Loans

Note that if segment j trades $X_j \to Y_j = X_{j+1}$, then, in order to not have to wait for any asset to be bridged, it will need to borrow X_j, swap to Y_j, and bridge $Y_j = X_{j+1}$ to chain $B_{i_{j+1}}$. The bridged asset will then allow segment $j+1$ to settle its debt, and this scheme will be repeated along the entire cycle so that each segment will bridge the asset required by the successive segment to repay its loan.

Since flash loans cannot possibly span multi-chain bridges, each segment j draws capital from a single collateral vault on its home chain B_{i_j}. Execution for segment j will therefore occurs in at least two main transactions:

1. **Borrow–Swap–Bridge (SIA style):**
   (a) Borrow asset X_j against collateral in B_{i_j}.
   (b) Swap $X_j \to Y_j$ on the local DEX.
   (c) Bridge Y_j to chain $B_{i_{j+1}}$.
2. **Debt Settlement:**
   (a) Receive bridged $Y_{j-1} = X_j$ from segment $j-1$.
   (b) Repay the loan for X_j, releasing collateral.

The final segment $c^{(r)}$ returns any surplus back to the initial vault.

Practical Considerations

- **Upfront Capital Requirement:** Although this strategy greatly reduces the capital requirements that we have seen in regular SIA, we still require around b large stashes of collateral (one per chain). Because the loan size is generally in the same order of magnitude as the pledged collateral, this implies that no trade can be significantly greater in size than the fraction $1/b$ of the *total* upfront capital requirement.
- **Collateral Splitting:** If two distinct segments share chain B_i, they compete for the same vault, reducing per-segment leverage. Since the overall leverage is related to the minimum of the maximum leverage possible on each segment, it follows that two

distinct segments sharing the same chain are already sufficient to roughly halve the maximum leverage available to the combined trade.

- **Timing Drift:** Parallel execution mitigates bridge delays but cannot eliminate differences in block times or network latency, reintroducing pseudo-atomic risk.
- **Increased Gas Costs:** Note that for each trade we require at least $2r$ transactions in total, which means that gas fees will rapidly balloon. This is partially mitigated by the substantially lower L2 gas fees, but even for the canonical 2-chain example we would already have 4 transactions to execute instead of one as in the atomic case.
- **Gas Competition:** We assume cross-chain segments are dispersed enough to avoid intense gas wars; adding gas-bidding per segment would hugely increase costs and complexity, since we would be required to "win" the bids for every single section of the cycle.

4 Experimental Evaluation & Discussion

For a preliminary analyses of our framework, we run a simplified implementation of the pseudocode found in section 3.2 on a snapshot of the global state of the Uniswap V2 DEX.[2] The goal here is to focus in depth on part 2 of the algorithm (the offline data processing, graph pruning, and modified Bellman-Ford cycle-detection routine) by operating on a frozen snapshot in time of all the DEX pairs, and assuming that the potential arbitrage transactions detected are submitted to Flashbots by a separate routine. This approach allows us to experiment in depth with the behavior of this particular module independently of the development of the other modules.

4.1 End-to-End Implementation in Python

Our Python program implements the single-chain arbitrage pipeline in four stages. First, it retrieves a complete snapshot of all Uniswap V2 pools via TheGraph: it queries the factory contract for the total pair count, then pages through every liquidity pool (in batches of size `PAGE_SIZE`), fetching reserves, token IDs, prices, volumes, and other metadata. This raw data is written to a JSON file so that subsequent steps can run offline against a fixed snapshot.

In the second stage, the program applies aggressive filtering and graph construction. Zero-reserve pairs, blacklisted tokens, and pools below a minimum reserve threshold are discarded. The remaining pools are loaded into a custom `Graph` class, where each pair yields two directed `Edge` objects (forward and backward) whose weights incorporate the fixed

[2] Our code implementation can be found in the following GitHub repository: `https://github.com/Roby36/BCP.git`.

Uniswap V2 0.3% fee. The graph is then pruned by minimum out-degree and other heuristics, reducing node count from hundreds of thousands to a few thousand well-connected tokens. Finally, each rate is transformed via a negative logarithm to produce the weighted graph G', which is thus ready to be fed into the Bellman-Ford routine in the next stage.

The third stage runs a modified Bellman–Ford on G' as described in the preceding section. A helper routine `enumerate_cycles` repeatedly finds one negative cycle, removes its first edge, and re-runs the search to collect a list of distinct loops. Thanks to the effective pruning, the routine is able to enumerate all the negative cycles within an acceptable time.

The fourth stage maps each raw log-cycle back to its original `Edge` objects, and then recovers the swap data for each cycle. This is then fed into the `router_swap` routine that fetches the Uniswap V2 router contract's ABI and simulates the entire exchange sequence with an optimal amount Δx^* locally through the `.call()` interface of the Python Web3 client, in order to confirm that the final output amount is greater than the input amount as alleged by the negative cycle detection algorithm.

This routine allows us to simulate changes on-chain, and thus preemptively discard non-profitable trades to save on gas fees.

4.2 Results and Discussion

In the Appendix we include the output of the program described above, in the form of a list of the cycles detected, where each cycle is represented by a list of its adjacent liquidity pools (note that each liquidity pool shares exactly one token), along with a list of the products for each of the cycles.

5 Conclusion

In conclusion, this work addresses gaps between theory and practice in blockchain arbitrage detection and execution. First, whereas prior detection algorithms have largely remained at a theoretical level, we offer detailed, pragmatic guidance for embedding these methods within a full-fledged, architecture-aware trading system. We move beyond abstract formulation to demonstrate how arbitrage detection can be incorporated into a multithreaded, single-chain environment.

To that end, we deliver an end-to-end implementation composed of four tightly integrated modules:

1. **Online Listener** — Continuously ingests and maintains the latest blockchain state.
2. **Offline Data Processor** — Transforms raw state data into a compact, graph-based representation optimized for fast querying.

3. **Offline Solver** — Executes our core detection algorithms to uncover profitable cyclic trades
4. **Online Executor** — Submits crafted transactions to the network, dynamically adjusting gas bids to ensure timely inclusion.

Finally, we propose a way to extend the single-chain framework to a broader multi-chain setting. By generalizing our detection principles to operate across interconnected ledgers, we pave the way for identifying cross-chain arbitrage paths that may yield even greater returns. In doing so, we lay both the theoretical groundwork and practical scaffolding for future systems capable of exploiting complex, multi-chain opportunities. Together, these contributions chart a concrete path from algorithmic insight to deployable trading infrastructure and set the stage for ongoing innovation in decentralized finance.

Our research here is by no means complete, and we encourage future work to explore some of the following areas of inquiry.

Within the single-chain, canonical Uniswap V2 case, we have relied on *TheGraph*'s API to supply us with the up-to-date list of liquidity pools on the DEX. While this was a necessary intermediate step, these queries are slow and crucially not as up-to-date as direct querying via an Ethereum full node. Hence, future research could implement a full-node client for the first module (online listener) that directly query the factory contract at each block to directly track the evolving states of the liquidity pools on the DEX.

Moreover, whilst the second and third modules are much more fully implemented than the first, future research could aim at refining the specific filtering parameters to increase efficiency. Finally, there needs to be a full implemenation of the fourth module interfaces the cycle-detection logic with Flashbots to execute transactions. The advantage of this approach is that it allows us to submit the transaction to a "builder" that verifies its validity off-chain, and then only executes it on-chain for a fee if profitable, thereby saving on significant gas expenses that would have been used to execute unprofitable transactions.

Lastly, shifting away from the simpler V2 architecture, future research should address the case of Uniswap V3, where for any two given tokens there are several different liquidity pools at different fee tiers. This greatly enhances the search space in the graph because it opens a variety of options to trade between any two given tokens at given sizes. More generally, a more developed system will also take into account several other DEXs, with each one having its dedicated online listener routine.

Another aspect to further articulate in code is the liquidity provision in both the single-chain setting with flash loans and the multi-chain setting with collateralized loans, as the last part of the offline component described above.

Finally, further research needs to be done to test the practical viability of the extended multi-chain strategy that we have framed theoretically. This will likely require adjustments

to a well-developed single-chain system, as the one we have proposed.

References

Angeris, Guillermo et al. (2021). *Optimal Routing for Constant Function Market Makers*. https://arxiv.org/abs/2204.05238. arXiv preprint arXiv:2204.05238, accessed May 6, 2025.

DeFigueiredo, Dimitri (2022). *Crypto Arbitrage Cycles*. https://dimitri.xyz/crypto-arbitrage-cycles/. Accessed: 2025-05-06.

Johnson, Donald B. (1975). "Finding All the Elementary Circuits of a Directed Graph". In: *SIAM Journal on Computing* 4.1, pp. 77–84. DOI: 10.1137/0204007.

Lipton, Alex and Marcos López de Prado (Feb. 2020). "A Closed-Form Solution for Optimal Mean-Reverting Trading Strategies". In: Available at SSRN: https://ssrn.com/abstract=3534445. DOI: 10.2139/ssrn.3534445. URL: http://dx.doi.org/10.2139/ssrn.3534445.

McKinnon, Daniel D. (2022). *All is Fair in Arb and MEV on Avalanche C-Chain*. https://www.ddmckinnon.com/2022/11/27/all-is-fair-in-arb-and-mev-on-avalanche-c-chain/. Accessed: 2025-05-06.

McLaughlin, R., C. Kruegel, and G. Vigna (2023). "A Large Scale Study of the Ethereum Arbitrage Ecosystem". In: *Proceedings of the 32nd USENIX Security Symposium (USENIX Security '23)*, pp. 3295–3312.

Öz, Burak et al. (2025). "Pandora's Box: Cross-Chain Arbitrages in the Realm of Blockchain Interoperability". In: *arXiv preprint arXiv:2501.17335*. DOI: 10.48550/arXiv.2501.17335. URL: https://arxiv.org/abs/2501.17335.

Wang, Y. et al. (2022). "Cyclic Arbitrage in Decentralized Exchanges". In: *Companion Proceedings of the Web Conference 2022*, pp. 12–19.

Zhang, Yu, Zichen Li, et al. (2024). "Profit Maximization in Arbitrage Loops". In: *2024 IEEE 44th International Conference on Distributed Computing Systems Workshops (ICDCSW)*. IEEE, pp. 153–160. DOI: 10.1109/ICDCSW63686.2024.00028.

Zhang, Yu, Tao Yan, et al. (2024). "An Improved Algorithm to Identify More Arbitrage Opportunities on Decentralized Exchanges". In: *2024 IEEE International Conference on Blockchain and Cryptocurrency (ICBC)*, pp. 1–7. DOI: 10.1109/ICBC59979.2024.10634351.

Zhou, Liyi et al. (2021). "On the Just-In-Time Discovery of Profit-Generating Transactions in DeFi Protocols". In: *2021 IEEE Symposium on Security and Privacy (SP)*, pp. 919–936. DOI: 10.1109/SP40001.2021.00113.

A The products of each of the profitable cycles

Below is a JSON dump of products of the seven profitable cycles (i.e., products above 1) that our algorithm detected, organized in descending order from the product of the most profitable cycle:

Listing 1: The products of each of the profitable cycles


```
[
    25.331522796299595,
    16.670958057849646,
    7.17936083745079,
    5.366899173041375,
    1.0223289520448882,
    1.0089776340598704,
    1.0000119043223585
]
```

B JSON output of profitable cycles detected by our algorithm

Below is the full JSON dump of the cycles in our snapshot with a product above one:

Listing 2: Profitable Cycles Detected By Our Algorithm


```
[
    [
        {
            "createdAtBlockNumber": "10710767",
            "createdAtTimestamp": "1598108961",
            "id": "0x582e3da39948c6339433008703211ad2c13eb2ac
                ",
            "liquidityProviderCount": "105",
            "reserve0": "1938.71572355778713683",
            "reserve1": "679288.231504582",
            "reserveUSD": "
                1500508.669051997961563828480215194",
            "token0": {
                "decimals": "18",
                "id": "0
                    xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                    ,
                "symbol": "WETH"
            },
            "token0Price": "
                0.002854039910663033363561726861932482",
            "token1": {
                "decimals": "9",
                "id": "0
                    xd233d1f6fd11640081abb8db125f722b5dc729dc"
                    ,
                "symbol": "USD"
            },
            "token1Price": "
                350.380524204963210381266949061591 8",
            "totalSupply": "0.817877991073295535",
            "trackedReserveETH": "
                3877.43144711557427366000000000001",
            "txCount": "2965",
            "untrackedVolumeUSD": "
                8176945.48348262307671796041150846 5",
            "volumeUSD": "8153223.199450599065709993479126284
                "
        },
        {
```

```
            "createdAtBlockNumber": "10483166",
            "createdAtTimestamp": "1595071291",
            "id": "0x2fdbadf3c4d5a8666bc06645b8358ab803996e28",
            "liquidityProviderCount": "4497",
            "reserve0": "24.89569142153712159",
            "reserve1": "71.911089137678191769",
            "reserveUSD": "261313.048719132361927090677442632 3",
            "token0": {
                "decimals": "18",
                "id": "0x0bc529c00c6401aef6d220be8c6ea1667f6ad93e",
                "symbol": "YFI"
            },
            "token0Price": "0.346201006271408197669545659356843 2",
            "token1": {
                "decimals": "18",
                "id": "0xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2",
                "symbol": "WETH"
            },
            "token1Price": "2.888495359300136408487222205318017",
            "totalSupply": "15.451905643220682298",
            "trackedReserveETH": "143.822178275356383537999999999999 99",
            "txCount": "302416",
            "untrackedVolumeUSD": "2546076092.66079082502984788294247",
            "volumeUSD": "2546065045.541785603304438664186908"
        },
        {
            "createdAtBlockNumber": "10713805",
            "createdAtTimestamp": "1598149012",
            "id": "0x485d20e8aac9f7c6a39d05147737487ff4093af4",
            "liquidityProviderCount": "12",
            "reserve0": "1.085106549156937531",
            "reserve1": "28071.126412673",
            "reserveUSD": "62611.853156904765007598080775181 58",
```

```
            "token0": {
                "decimals": "18",
                "id": "0
                   x0bc529c00c6401aef6d220be8c6ea1667f6ad93e"
                   ,
                "symbol": "YFI"
            },
            "token0Price": "
               0.000038655611221466872451201328544690 16",
            "token1": {
                "decimals": "9",
                "id": "0
                   xd233d1f6fd11640081abb8db125f722b5dc729dc"
                   ,
                "symbol": "USD"
            },
            "token1Price": "
               25869.465477360334279664671658722 76",
            "totalSupply": "0.005449251206479078",
            "trackedReserveETH": "
               169.328481512345533531064796020831 2",
            "txCount": "150",
            "untrackedVolumeUSD": "
               134579.313033580015796442151439334 1",
            "volumeUSD": "118753.549514684861517978506574426 3
               "
        }
    ],
    [
        {
            "createdAtBlockNumber": "10091443",
            "createdAtTimestamp": "1589824355",
            "id": "0xc2adda861f89bbb333c90c492cb837741916a225
               ",
            "liquidityProviderCount": "1447",
            "reserve0": "376.903428049533234204",
            "reserve1": "320.636404769408452345",
            "reserveUSD": "1167447.433675655437011828629307 78
               ",
            "token0": {
                "decimals": "18",
                "id": "0
                   x9f8f72aa9304c8b593d555f12ef6589cc3a579a2"
                   ,
                "symbol": "MKR"
            },
```

```
            "token0Price": "
               1.175485448449280870049211647056836",
            "token1": {
                "decimals": "18",
                "id": "0
                   xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                   ,
                "symbol": "WETH"
            },
            "token1Price": "
               0.850712360003448728985058771764413 2",
            "totalSupply": "229.432034755976158814",
            "trackedReserveETH": "
               641.272809538816904690000000000003",
            "txCount": "181386",
            "untrackedVolumeUSD": "
               1584054597.158094409841552187712457",
            "volumeUSD": "1584054570.15949107061620405125324 2
               "
        },
        {
            "createdAtBlockNumber": "10483166",
            "createdAtTimestamp": "1595071291",
            "id": "0x2fdbadf3c4d5a8666bc06645b8358ab803996e28
               ",
            "liquidityProviderCount": "4497",
            "reserve0": "24.89569142153712159",
            "reserve1": "71.911089137678191769",
            "reserveUSD": "
               261313.048719132361927090677442632 3",
            "token0": {
                "decimals": "18",
                "id": "0
                   x0bc529c00c6401aef6d220be8c6ea1667f6ad93e"
                   ,
                "symbol": "YFI"
            },
            "token0Price": "
               0.346201006271408197669545659356843 2",
            "token1": {
                "decimals": "18",
                "id": "0
                   xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                   ,
                "symbol": "WETH"
            },
```

```
            "token1Price": "
               2.888495359300136408487222205318017",
            "totalSupply": "15.451905643220682298",
            "trackedReserveETH": "
               143.82217827535638353799999999999999",
            "txCount": "302416",
            "untrackedVolumeUSD": "
               2546076092.66079082502984788294247",
            "volumeUSD": "2546065045.541785603304438664186908
               "
        },
        {
            "createdAtBlockNumber": "10713805",
            "createdAtTimestamp": "1598149012",
            "id": "0x485d20e8aac9f7c6a39d05147737487ff4093af4
               ",
            "liquidityProviderCount": "12",
            "reserve0": "1.085106549156937531",
            "reserve1": "28071.126412673",
            "reserveUSD": "
               62611.85315690476500759808077518158",
            "token0": {
                "decimals": "18",
                "id": "0
                   x0bc529c00c6401aef6d220be8c6ea1667f6ad93e"
                   ,
                "symbol": "YFI"
            },
            "token0Price": "
               0.000038655611221466872451201328544690 16",
            "token1": {
                "decimals": "9",
                "id": "0
                   xd233d1f6fd11640081abb8db125f722b5dc729dc"
                   ,
                "symbol": "USD"
            },
            "token1Price": "
               25869.465477360334279664671658722 76",
            "totalSupply": "0.005449251206479078",
            "trackedReserveETH": "
               169.32848151234553353106479602083 12",
            "txCount": "150",
            "untrackedVolumeUSD": "
               134579.31303358001579644215143933 41",
            "volumeUSD": "118753.549514684861517978506574426 3
```

```
                "
        },
        {
            "createdAtBlockNumber": "10732479",
            "createdAtTimestamp": "1598396944",
            "id": "0xa16a3cbc92d77b720a851d33a91890c4fbfb0299
                ",
            "liquidityProviderCount": "15",
            "reserve0": "34.196702946558763307",
            "reserve1": "15441.974611678",
            "reserveUSD": "
                32766.562887157381277140676860872 97",
            "token0": {
                "decimals": "18",
                "id": "0
                    x9f8f72aa9304c8b593d555f12ef6589cc3a579a2"
                    ,
                "symbol": "MKR"
            },
            "token0Price": "
                0.00221452915229490740559177395081 9688",
            "token1": {
                "decimals": "9",
                "id": "0
                    xd233d1f6fd11640081abb8db125f722b5dc729dc"
                    ,
                "symbol": "USD"
            },
            "token1Price": "
                451.563258475827303528911115989178 6",
            "totalSupply": "0.022728673123465121",
            "trackedReserveETH": "
                88.859567625217695690256209287702 35",
            "txCount": "149",
            "untrackedVolumeUSD": "
                90463.178353342592293202811072251 36",
            "volumeUSD": "72853.597190161589344813286707509 26
                "
        }
    ],
    [
        {
            "createdAtBlockNumber": "10091428",
            "createdAtTimestamp": "1589824102",
            "id": "0xa2107fa5b38d9bbd2c461d6edf11b11a50f6b974
                ",
```

```
            "liquidityProviderCount": "8020",
            "reserve0": "51677.458244523991696657",
            "reserve1": "388.37670996821168704",
            "reserveUSD": "
               1419705.51193485266964074939458594 8",
            "token0": {
                "decimals": "18",
                "id": "0
                   x514910771af9ca656af840dff83e8264ecf986ca"
                   ,
                "symbol": "LINK"
            },
            "token0Price": "
               133.06013702200049270413983904275 15",
            "token1": {
                "decimals": "18",
                "id": "0
                   xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                   ,
                "symbol": "WETH"
            },
            "token1Price": "
               0.007515398844318471749032322762847439",
            "totalSupply": "2381.354985633405696159",
            "trackedReserveETH": "
               776.75341993642337408000000000000001",
            "txCount": "499665",
            "untrackedVolumeUSD": "
               4270386360.471612493335722993195219",
            "volumeUSD": "4270385958.71816236558828410980358"
        },
        {
            "createdAtBlockNumber": "10483166",
            "createdAtTimestamp": "1595071291",
            "id": "0x2fdbadf3c4d5a8666bc06645b8358ab803996e28
               ",
            "liquidityProviderCount": "4497",
            "reserve0": "24.89569142153712159",
            "reserve1": "71.911089137678191769",
            "reserveUSD": "
               261313.048719132361927090677442 6323",
            "token0": {
                "decimals": "18",
                "id": "0
                   x0bc529c00c6401aef6d220be8c6ea1667f6ad93e"
                   ,
```

```
                "symbol": "YFI"
            },
            "token0Price": "
               0.346201006271408197669545659356843 2",
            "token1": {
                "decimals": "18",
                "id": "0
                   xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                   ,
                "symbol": "WETH"
            },
            "token1Price": "
               2.888495359300136408487222205318017",
            "totalSupply": "15.451905643220682298",
            "trackedReserveETH": "
               143.82217827535638353799999999999999",
            "txCount": "302416",
            "untrackedVolumeUSD": "
               2546076092.66079082502984788294247",
            "volumeUSD": "2546065045.541785603304438664186908
               "
        },
        {
            "createdAtBlockNumber": "10713805",
            "createdAtTimestamp": "1598149012",
            "id": "0x485d20e8aac9f7c6a39d05147737487ff4093af4
               ",
            "liquidityProviderCount": "12",
            "reserve0": "1.085106549156937531",
            "reserve1": "28071.126412673",
            "reserveUSD": "
               62611.85315690476500759808077518158",
            "token0": {
                "decimals": "18",
                "id": "0
                   x0bc529c00c6401aef6d220be8c6ea1667f6ad93e"
                   ,
                "symbol": "YFI"
            },
            "token0Price": "
               0.000038655611221466872451201328544690 16",
            "token1": {
                "decimals": "9",
                "id": "0
                   xd233d1f6fd11640081abb8db125f722b5dc729dc"
                   ,
```

```
            "symbol": "USD"
        },
        "token1Price": "
           25869.465477360334279664671658722 76",
        "totalSupply": "0.005449251206479078",
        "trackedReserveETH": "
           169.32848151234553353106479602083 12",
        "txCount": "150",
        "untrackedVolumeUSD": "
           134579.313033580015796442151439334 1",
        "volumeUSD": "118753.549514684861517978506574426 3
           "
    },
    {
        "createdAtBlockNumber": "10732489",
        "createdAtTimestamp": "1598397120",
        "id": "0x81a8bd7f2b29cee72aae18da9b4637acf4bc125a
           ",
        "liquidityProviderCount": "12",
        "reserve0": "2452.999633978536673792",
        "reserve1": "22722.711254473",
        "reserveUSD": "
           52508.536521275120548981685277731 54",
        "token0": {
            "decimals": "18",
            "id": "0
               x514910771af9ca656af840dff83e8264ecf986ca"
               ,
            "symbol": "LINK"
        },
        "token0Price": "
           0.107953650711275046201208349128759 7",
        "token1": {
            "decimals": "9",
            "id": "0
               xd233d1f6fd11640081abb8db125f722b5dc729dc"
               ,
            "symbol": "USD"
        },
        "token1Price": "
           9.263234669798495301144892168490241",
        "totalSupply": "0.233800042414231333",
        "trackedReserveETH": "
           141.669155436347477936006106057902 1",
        "txCount": "111",
        "untrackedVolumeUSD": "
```

```
                79920.893792848537272791130740340 34",
            "volumeUSD": "50177.485560482342962889071959916 03
                "
        }
    ],
    [
        {
            "createdAtBlockNumber": "10483166",
            "createdAtTimestamp": "1595071291",
            "id": "0x2fdbadf3c4d5a8666bc06645b8358ab803996e28
                ",
            "liquidityProviderCount": "4497",
            "reserve0": "24.89569142153712159",
            "reserve1": "71.911089137678191769",
            "reserveUSD": "
                261313.048719132361927090677442632 3",
            "token0": {
                "decimals": "18",
                "id": "0
                    x0bc529c00c6401aef6d220be8c6ea1667f6ad93e"
                    ,
                "symbol": "YFI"
            },
            "token0Price": "
                0.346201006271408197669545659356843 2",
            "token1": {
                "decimals": "18",
                "id": "0
                    xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                    ,
                "symbol": "WETH"
            },
            "token1Price": "
                2.888495359300136408487222205318017",
            "totalSupply": "15.451905643220682298",
            "trackedReserveETH": "
                143.822178275356383537999999999999 9",
            "txCount": "302416",
            "untrackedVolumeUSD": "
                2546076092.660790825029847882942 47",
            "volumeUSD": "2546065045.541785603304438664186908
                "
        },
        {
            "createdAtBlockNumber": "10713805",
            "createdAtTimestamp": "1598149012",
```

```
            "id": "0x485d20e8aac9f7c6a39d05147737487ff4093af4
               ",
            "liquidityProviderCount": "12",
            "reserve0": "1.085106549156937531",
            "reserve1": "28071.126412673",
            "reserveUSD": "
               62611.85315690476500759808077518158",
            "token0": {
                "decimals": "18",
                "id": "0
                   x0bc529c00c6401aef6d220be8c6ea1667f6ad93e"
                   ,
                "symbol": "YFI"
            },
            "token0Price": "
               0.000038655611221466872451201328544690 16",
            "token1": {
                "decimals": "9",
                "id": "0
                   xd233d1f6fd11640081abb8db125f722b5dc729dc"
                   ,
                "symbol": "USD"
            },
            "token1Price": "
               25869.465477360334279664671658722 76",
            "totalSupply": "0.005449251206479078",
            "trackedReserveETH": "
               169.32848151234553353106479602083 12",
            "txCount": "150",
            "untrackedVolumeUSD": "
               134579.31303358001579644215143933 41",
            "volumeUSD": "118753.549514684861517978506574426 3
               "
        },
        {
            "createdAtBlockNumber": "10718794",
            "createdAtTimestamp": "1598214744",
            "id": "0xf6c0428f56f9ca7ed54bd824a4856e92c3a8e401
               ",
            "liquidityProviderCount": "26",
            "reserve0": "17738.720438",
            "reserve1": "16115.40615793",
            "reserveUSD": "
               34670.391036396976026232372991774 73",
            "token0": {
                "decimals": "6",
```

```
                "id": "0
                    xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48"
                    ,
                "symbol": "USDC"
            },
            "token0Price": "
                1.100730584396174617401022516829952",
            "token1": {
                "decimals": "9",
                "id": "0
                    xd233d1f6fd11640081abb8db125f722b5dc729dc"
                    ,
                "symbol": "USD"
            },
            "token1Price": "
                0.908487521084523898284573664354403",
            "totalSupply": "0.000000528636816933",
            "trackedReserveETH": "
                95.697605880280321964990833251935393",
            "txCount": "228",
            "untrackedVolumeUSD": "
                174832.560407407494299066133496033 2",
            "volumeUSD": "173849.329623893974174332864048960
                2"
        },
        {
            "createdAtBlockNumber": "10970025",
            "createdAtTimestamp": "1601553315",
            "id": "0x15e470f37621b6a635d32c3d8473a578b5ec0d2b
                ",
            "liquidityProviderCount": "140",
            "reserve0": "9283.443696",
            "reserve1": "1476729.023481286836740919",
            "reserveUSD": "
                18615.012901700035190271877164043 15",
            "token0": {
                "decimals": "6",
                "id": "0
                    xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48"
                    ,
                "symbol": "USDC"
            },
            "token0Price": "
                0.006286490986758641526870454762136874",
            "token1": {
                "decimals": "18",
```

```
                "id": "0
                    xcf9c692f7e62af3c571d4173fd4abf9a3e5330d0"
                    ,
                "symbol": "ONIGIRI"
            },
            "token1Price": "
                159.071253280457967323349078886900 2",
            "totalSupply": "0.109609222122791857",
            "trackedReserveETH": "
                10.221387739747350447646234892820 8",
            "txCount": "3451",
            "untrackedVolumeUSD": "
                1026946.075336327002170620497779228",
            "volumeUSD": "1023165.737207681493189588419975739
                "
        },
        {
            "createdAtBlockNumber": "10969832",
            "createdAtTimestamp": "1601550702",
            "id": "0x9866103b576bc362e934b8e96115e72413b6a8c2
                ",
            "liquidityProviderCount": "251",
            "reserve0": "22.420388915558486958",
            "reserve1": "6453332.09190381921499603",
            "reserveUSD": "
                81050.064623487545422970993035841 99",
            "token0": {
                "decimals": "18",
                "id": "0
                    xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                    ,
                "symbol": "WETH"
            },
            "token0Price": "
                0.000003474234487899130041909209100588 29",
            "token1": {
                "decimals": "18",
                "id": "0
                    xcf9c692f7e62af3c571d4173fd4abf9a3e5330d0"
                    ,
                "symbol": "ONIGIRI"
            },
            "token1Price": "
                287833.191306756068890952499261632 6",
            "totalSupply": "11000.70667649802242041",
            "trackedReserveETH": "
```

```
                44.840777831116973915999999999999999",
421             "txCount": "6800",
422             "untrackedVolumeUSD": "
                3717915.048393718924516267350785558",
423             "volumeUSD": "3713876.444622676073411174791779623
                "
424         }
425     ],
426     [
427         {
428             "createdAtBlockNumber": "10720632",
429             "createdAtTimestamp": "1598239311",
430             "id": "0x1d1126cc2c77384448913b41fbf308563aae1f16
                ",
431             "liquidityProviderCount": "46",
432             "reserve0": "34842.821405583194634723",
433             "reserve1": "32690.980706976",
434             "reserveUSD": "
                72151.204336059249357143434143345 32",
435             "token0": {
436                 "decimals": "18",
437                 "id": "0
                    x6b175474e89094c44da98b954eedeac495271d0f"
                    ,
438                 "symbol": "DAI"
439             },
440             "token0Price": "
                1.065823681396869462325381955605209",
441             "token1": {
442                 "decimals": "9",
443                 "id": "0
                    xd233d1f6fd11640081abb8db125f722b5dc729dc"
                    ,
444                 "symbol": "USD"
445             },
446             "token1Price": "
                0.938241491021666091060390594679086 7",
447             "totalSupply": "1.055061368743852374",
448             "trackedReserveETH": "
                189.475333506786817231255911913888 5",
449             "txCount": "248",
450             "untrackedVolumeUSD": "
                136733.770228399157395136543498470 2",
451             "volumeUSD": "137852.125192505934069527397218261 5
                "
452         },
```

```
        {
            "createdAtBlockNumber": "10718794",
            "createdAtTimestamp": "1598214744",
            "id": "0xf6c0428f56f9ca7ed54bd824a4856e92c3a8e401
               ",
            "liquidityProviderCount": "26",
            "reserve0": "17738.720438",
            "reserve1": "16115.40615793",
            "reserveUSD": "
               34670.391036396976026232372991774 73",
            "token0": {
                "decimals": "6",
                "id": "0
                   xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48"
                   ,
                "symbol": "USDC"
            },
            "token0Price": "
               1.100730584396174617401022516829952",
            "token1": {
                "decimals": "9",
                "id": "0
                   xd233d1f6fd11640081abb8db125f722b5dc729dc"
                   ,
                "symbol": "USD"
            },
            "token1Price": "
               0.908487521084523898284573664354403",
            "totalSupply": "0.000000528636816933",
            "trackedReserveETH": "
               95.697605880280321964990832519353 93",
            "txCount": "228",
            "untrackedVolumeUSD": "
               174832.560407407494299066133496033 2",
            "volumeUSD": "173849.32962389397417433286404896 02
               "
        },
        {
            "createdAtBlockNumber": "10060832",
            "createdAtTimestamp": "1589413139",
            "id": "0xae461ca67b15dc8dc81ce7615e0320da1a9ab8d5
               ",
            "liquidityProviderCount": "1555",
            "reserve0": "735425.668664721050525053",
            "reserve1": "736256.712194",
            "reserveUSD": "
```

```
                1471594.889914583443271149954729399",
            "token0": {
                "decimals": "18",
                "id": "0
                    x6b175474e89094c44da98b954eedeac495271d0f"
                    ,
                "symbol": "DAI"
            },
            "token0Price": "
                0.998871258467983958266806418053054 8",
            "token1": {
                "decimals": "6",
                "id": "0
                    xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48"
                    ,
                "symbol": "USDC"
            },
            "token1Price": "
                1.001130017029168749884055922180115",
            "totalSupply": "0.648859715870591849",
            "trackedReserveETH": "
                809.942198151827878616109558016564 4",
            "txCount": "183774",
            "untrackedVolumeUSD": "
                577361271.672800001902053573712817 4",
            "volumeUSD": "577361271.672800001902053573712817 4
                "
        }
    ],
    [
        {
            "createdAtBlockNumber": "11077058",
            "createdAtTimestamp": "1602985820",
            "id": "0xff84179bf75737b9b651d712d6809e44cffb382a
                ",
            "liquidityProviderCount": "50",
            "reserve0": "1134.814226060509738213",
            "reserve1": "3079.000955557393652657",
            "reserveUSD": "
                30975.725436263153133118478366355 43",
            "token0": {
                "decimals": "18",
                "id": "0
                    x05a0d55bbb2c047a3624af4db6d3a3ce2ea7da09"
                    ,
                "symbol": "Penguin"
```

```
            },
            "token0Price": "
               0.368565727143489483365979399675038 7",
            "token1": {
                "decimals": "18",
                "id": "0
                   x1f9840a85d5af5bf1d1762f925bdaddc4201f984"
                   ,
                "symbol": "UNI"
            },
            "token1Price": "
               2.713220265352240486176571365918386",
            "totalSupply": "1787.115772085481421493",
            "trackedReserveETH": "
               16.780865554198562763673304890897 18",
            "txCount": "1664",
            "untrackedVolumeUSD": "
               739746.99145644149202667560615468 04",
            "volumeUSD": "664830.283338033386226867689731044
               8"
        },
        {
            "createdAtBlockNumber": "11076983",
            "createdAtTimestamp": "1602984703",
            "id": "0x24d0753e0b1db16ae4acc6456b83a43af142495b
               ",
            "liquidityProviderCount": "11",
            "reserve0": "1224.276101278320179761",
            "reserve1": "9.226060918915411646",
            "reserveUSD": "
               33674.597122952460219519470579927 98",
            "token0": {
                "decimals": "18",
                "id": "0
                   x05a0d55bbb2c047a3624af4db6d3a3ce2ea7da09"
                   ,
                "symbol": "Penguin"
            },
            "token0Price": "
               132.697595651930991867858313620537 8",
            "token1": {
                "decimals": "18",
                "id": "0
                   xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                   ,
                "symbol": "WETH"
```

```
            },
            "token1Price": "
               0.007535931567464298494442914837293623",
            "totalSupply": "83.001067880944680079",
            "trackedReserveETH": "18.452121837830823292",
            "txCount": "6975",
            "untrackedVolumeUSD": "
               3114131.407335525898915037661315 52",
            "volumeUSD": "2279014.585798437654056123944841034
               "
        },
        {
            "createdAtBlockNumber": "10876348",
            "createdAtTimestamp": "1600302037",
            "id": "0xd3d2e2692501a5c9ca623199d38826e513033a17
               ",
            "liquidityProviderCount": "23096",
            "reserve0": "284712.250334491561011916",
            "reserve1": "776.715520455560586111",
            "reserveUSD": "2834903.355107920352632060684336 21
               ",
            "token0": {
                "decimals": "18",
                "id": "0
                   x1f9840a85d5af5bf1d1762f925bdaddc4201f984"
                   ,
                "symbol": "UNI"
            },
            "token0Price": "
               366.559239304889413191818634830 8017",
            "token1": {
                "decimals": "18",
                "id": "0
                   xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                   ,
                "symbol": "WETH"
            },
            "token1Price": "
               0.002728072007941504173500557725461945",
            "totalSupply": "5854.943854589923086601",
            "trackedReserveETH": "1553.431040911121172222",
            "txCount": "805206",
            "untrackedVolumeUSD": "
               8200643528.637543276017923702719055",
            "volumeUSD": "8200568551.922778382094950284089457
               "
```

```
        }
    ],
    [
        {
            "createdAtBlockNumber": "10969832",
            "createdAtTimestamp": "1601550702",
            "id": "0x9866103b576bc362e934b8e96115e72413b6a8c2
               ",
            "liquidityProviderCount": "251",
            "reserve0": "22.420388915558486958",
            "reserve1": "6453332.09190381921499603",
            "reserveUSD": "
               81050.06462348754542297099303584199",
            "token0": {
                "decimals": "18",
                "id": "0
                   xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                   ,
                "symbol": "WETH"
            },
            "token0Price": "
               0.000003474234487899130041909209100588 29",
            "token1": {
                "decimals": "18",
                "id": "0
                   xcf9c692f7e62af3c571d4173fd4abf9a3e5330d0"
                   ,
                "symbol": "ONIGIRI"
            },
            "token1Price": "
               287833.191306756068890952499261632 6",
            "totalSupply": "11000.70667649802242041",
            "trackedReserveETH": "
               44.840777831116973915999999999999999",
            "txCount": "6800",
            "untrackedVolumeUSD": "
               3717915.048393718924516267350785558",
            "volumeUSD": "3713876.444622676073411174791779623
               "
        },
        {
            "createdAtBlockNumber": "10794697",
            "createdAtTimestamp": "1599220518",
            "id": "0x724d5c9c618a2152e99a45649a3b8cf198321f46
               ",
            "liquidityProviderCount": "235",
```

```
            "reserve0": "17079824.904460920200564964",
            "reserve1": "10.790554886575550585",
            "reserveUSD": "
               39420.818548622688157448207128695 91",
            "token0": {
                "decimals": "18",
                "id": "0
                   x557b933a7c2c45672b610f8954a3deb39a51a8ca"
                   ,
                "symbol": "REVV"
            },
            "token0Price": "
               1582849.546107198237213616966338902",
            "token1": {
                "decimals": "18",
                "id": "0
                   xc02aaa39b223fe8d0a0e5c4f27ead9083c756cc2"
                   ,
                "symbol": "WETH"
            },
            "token1Price": "
               0.000000631771985189219719349604134634 5757",
            "totalSupply": "5228.571725126125667907",
            "trackedReserveETH": "21.58110977315110117",
            "txCount": "102697",
            "untrackedVolumeUSD": "
               333392239.84296915331946386111847 65",
            "volumeUSD": "331798584.11894486756741341713253 31
               "
        },
        {
            "createdAtBlockNumber": "11904357",
            "createdAtTimestamp": "1613963860",
            "id": "0xacbf3c973eeec20c573ec930ce31198afb4ad0d5
               ",
            "liquidityProviderCount": "3",
            "reserve0": "10317849.38393058693518588",
            "reserve1": "11937.771304",
            "reserveUSD": "
               23854.872366024936815999958074206 02",
            "token0": {
                "decimals": "18",
                "id": "0
                   x557b933a7c2c45672b610f8954a3deb39a51a8ca"
                   ,
                "symbol": "REVV"
```

```
            },
            "token0Price": "
               864.302818439265599051040423583 5745",
            "token1": {
                "decimals": "6",
                "id": "0
                   xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48"
                   ,
                "symbol": "USDC"
            },
            "token1Price": "
               0.001157001896402203873033879317876163",
            "totalSupply": "0.258198760647682977",
            "trackedReserveETH": "
               13.133313980709428590672360028184 89",
            "txCount": "15742",
            "untrackedVolumeUSD": "
               13084694.942147729620160682013197 63",
            "volumeUSD": "2906214.871971718384717380411855646
               "
        },
        {
            "createdAtBlockNumber": "10970025",
            "createdAtTimestamp": "1601553315",
            "id": "0x15e470f37621b6a635d32c3d8473a578b5ec0d2b
               ",
            "liquidityProviderCount": "140",
            "reserve0": "9283.443696",
            "reserve1": "1476729.023481286836740919",
            "reserveUSD": "
               18615.012901700035190271877164043 15",
            "token0": {
                "decimals": "6",
                "id": "0
                   xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48"
                   ,
                "symbol": "USDC"
            },
            "token0Price": "
               0.006286490986758641526870454762136874",
            "token1": {
                "decimals": "18",
                "id": "0
                   xcf9c692f7e62af3c571d4173fd4abf9a3e5330d0"
                   ,
                "symbol": "ONIGIRI"
```

```
            },
            "token1Price": "
               159.071253280457967323349078886900 2",
            "totalSupply": "0.109609222122791857",
            "trackedReserveETH": "
               10.221387739747350447646234892820 8",
            "txCount": "3451",
            "untrackedVolumeUSD": "
               1026946.075336327002170620497779228",
            "volumeUSD": "1023165.737207681493189588419975739
               "
        }
    ]
]
```